Dock: V1 distinction & future configuration

A short product note capturing the current workflow, the differentiation concern, and the proposed later .dock configuration.

PRODUCT IDEA	**Dock automates project setup around Docker. Docker remains the container engine; Dock should make the path from project description to a running service simpler.**

Current V1 workflow

Today, a project author provides a Dockerfile and a Dock YAML configuration. Dock then automates the remaining setup steps: resolving/fetching the project requirements, building the image/container, and starting the service.

AUTHOR PROVIDES	DOCK AUTOMATES
1. Dockerfile **2.** dock.yaml	**3.** Read configuration and identify requirements **4.** Build the image/container **5.** Start the service

V1 release gate: implement and successfully verify pulling/running a Docker image specified in dock.yaml before publishing the pip package as V1.

Why Dock can feel like a Docker wrapper

The current workflow still asks users to write Docker-specific setup files themselves. Dock automates the final commands, which is useful, but the visible setup can make Dock look like a thin wrapper around Docker. The similar names, **Dock** and **Docker**, can add to that confusion.

The intended distinction is about the user experience: Dock automates project setup; Docker supplies the container engine. A future higher-level project description could make Dock's role clearer.

Later direction: replace two authored files with one .dock file

TODAY	LATER (PROPOSED)
User writes: - Dockerfile - dock.yaml	User writes: - a higher-level .dock project config
Dock consumes dock.yaml and uses the Dockerfile to build/run the project.	Dock reads .dock and generates the required Dockerfile and dock.yaml, then runs the existing setup workflow.

Scope: .dock is a later direction; keep the current files through V1 image-pull verification, then revisit naming and positioning.